

Sección 5: Patrones que Organizan la Lógica

- Strategy: reemplazar el switch de comisiones por una interfaz con múltiples implementaciones
- Inyección de colecciones en Spring: seleccionar la estrategia correcta en runtime
- Template Method: formalizar el flujo validar → calcular → ejecutar → guardar
- Factory Method: centralizar decisiones de construcción en TransactionFactory
- Observer con ApplicationEventPublisher: desacoplar side effects sin que el service los conozca
- Decorator: enriquecer comportamiento sin modificar el service original

52. ¿Por qué patrones de diseño?

➔ "Un patrón sin un dolor es overengineering."

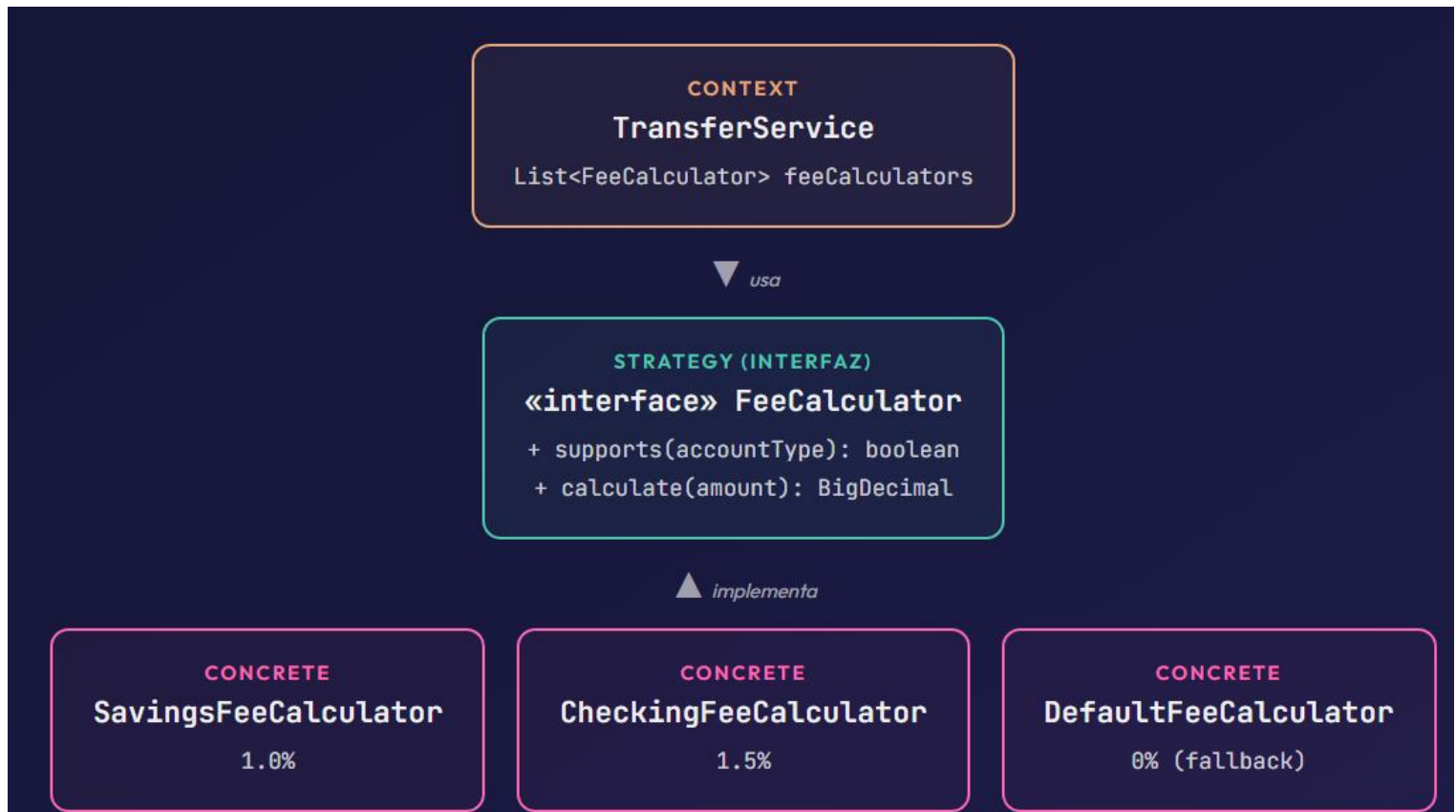
➔ ¿Qué son los patrones de diseño?

- Soluciones probadas a **problemas recurrentes**
- No son código que se copia y se pega
- Son **ideas** que se adaptan al contexto
- Los formalizaron en 1994 (**Gang of Four**)
- Spring Boot los usa internamente en todos lados

Patrón	Dolor en atlas-bank
Strategy	Switch de comisiones que crece con cada tipo de cuenta
Template Method	Flujo de transacción repetido con variaciones
Factory Method	Creación de transacciones acoplada al tipo
Observer	Side effects (notificaciones, auditoría) acoplados al service
Decorator	Agregar comportamiento (auditoría) sin modificar el service

53. Strategy en Spring: Inyección de colecciones

➔ Definir una familia de algoritmos, encapsular cada uno, y hacerlos intercambiables.



➔ ¿Cuándo usar Strategy?

- Tenés un comportamiento que **varía según el contexto**
- Querés agregar variantes **sin modificar código existente**
- Querés **eliminar switches** o cadenas de if/else

Si agregar una variante nueva significa crear una clase nueva y nada más → Strategy.

➔ Inyección de colecciones

```
TransferService.java  
  
private final List<FeeCalculator> feeCalculators;
```

Spring escanea el contexto, encuentra todos los **@Component** que implementen la interfaz, y los inyecta en la lista.

Si mañana agregás una implementación nueva → Spring la incluye **automáticamente**.

*Declarás un **List<Interfaz>** y Spring te inyecta todas las implementaciones.*

→ Auto-selección en runtime

```
BigDecimal fee = feeCalculators.stream()
    .filter(fc → fc.supports(from.getType())) // ¿vos manejas este tipo?
    .findFirst() // tomá la primera que diga sí
    .orElseThrow(...) // si ninguna → error
    .calculate(amount); // ejecutá el algoritmo
```

- El service **no tiene switches**. No sabe cuántas estrategias hay.
- Cada estrategia **se ofrece sola** cuando le corresponde.
- Agregar una variante nueva = crear una clase con **@Component**.

→ Controlando la inyección

@Order

Define la posición en la lista.

Menor número = mayor prioridad.

Útil para Strategy:
garantizar que el fallback quede al final.

@Primary

Marca un bean como opción por defecto.

Aplica cuando inyectás **uno solo**, no una lista.

No aplica a nuestro caso.

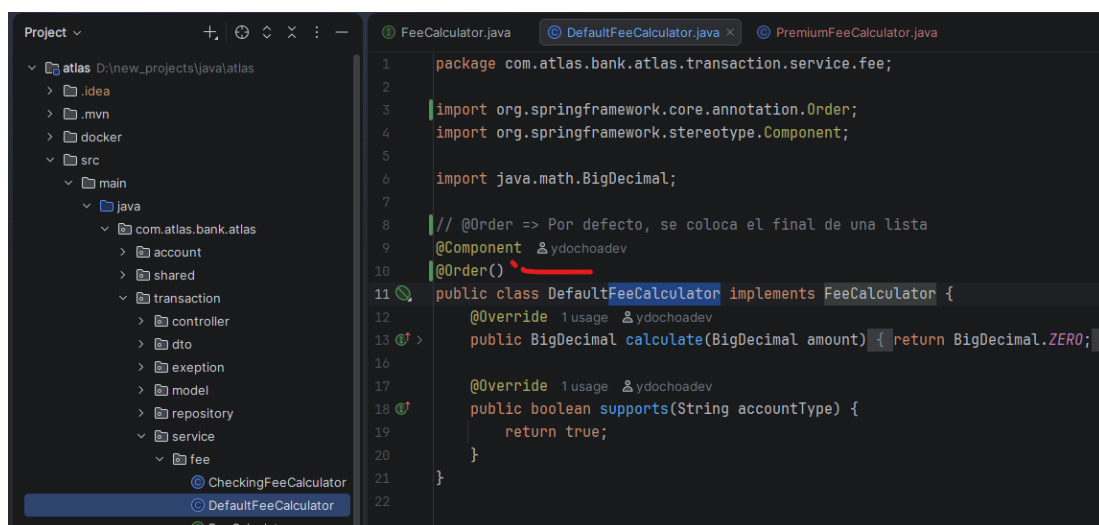
@Qualifier

Elegís un bean específico por nombre.

Opuesto a Strategy:
en Strategy, la selección es en runtime, no en compilación.

54. Strategy en el proyecto: Hands-on

→ Puede pasar que spring coloque a “DefaultFeeCalculator” (support siempre devuelve true) en primer lugar. El orden de los beans no está garantizado.



→ Para evitar errores, se colocará @Order a las demás implementaciones.

➔ Agregando otra cuenta:

```
FeeCalculator.java  DefaultFeeCalculator.java  PremiumFeeCalculator.java x
1  package com.atlas.bank.atlas.transaction.service.fee;
2
3  import org.springframework.core.annotation.Order;
4  import org.springframework.stereotype.Component;
5
6  import java.math.BigDecimal;
7
8  @Component
9  @Order(1)
10 public class PremiumFeeCalculator implements FeeCalculator {
11     @Override 1 usage
12     public BigDecimal calculate(BigDecimal amount) {
13         return BigDecimal.ZERO;
14     }
15
16     @Override 1 usage
17     public boolean supports(String accountType) {
18         return "PREMIUM".equals(accountType);
19     }
20 }
```

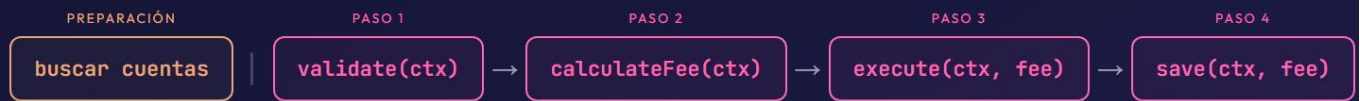
```
CheckingFeeCalculator.java x  FeeCalculator.java  DefaultFeeCalculator.java
1  package com.atlas.bank.atlas.transaction.service.fee;
2
3  import org.springframework.core.annotation.Order;
4  import org.springframework.stereotype.Component;
5
6  import java.math.BigDecimal;
7
8  @Component ydochoadev
9  @Order(1)
10 public class CheckingFeeCalculator implements FeeCalculator {
11     @Override 1 usage ydochoadev
12     public BigDecimal calculate(BigDecimal amount) { return am
13
14
15
16     @Override 1 usage ydochoadev
17     public boolean supports(String accountType) { return "CHEC
18
19
20 }
```

```
CheckingFeeCalculator.java  SavingsFeeCalculator.java x  FeeCalculator.java
1  package com.atlas.bank.atlas.transaction.service.fee;
2
3  import org.springframework.core.annotation.Order;
4  import org.springframework.stereotype.Component;
5
6  import java.math.BigDecimal;
7
8  @Component ydochoadev
9  @Order(1)
10 public class SavingsFeeCalculator implements FeeCalculat
11     @Override 1 usage ydochoadev
12     public BigDecimal calculate(BigDecimal amount) { ret
13
14
15
16     @Override 1 usage ydochoadev
17     public boolean supports(String accountType) { return
18
19
20 }
```

55. Template Method: parte 1

→ Definir el esqueleto de un algoritmo en una clase base, dejando que las subclases completen los pasos que varían.

→ El flujo en TransferService:



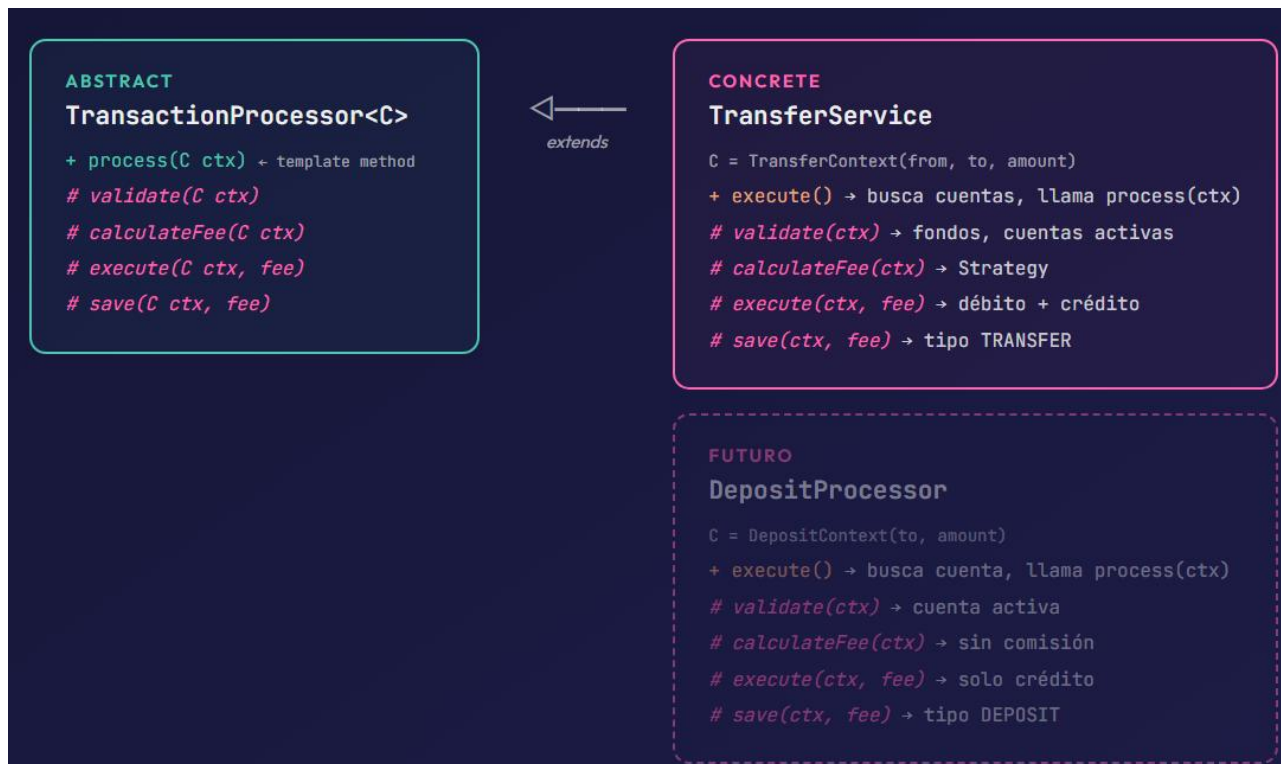
Buscar cuentas es preparación — no varía entre tipos de transacción.

Los cuatro pasos reciben un contexto con los datos de la operación. Sin estado en el service.

Transferencia, depósito, retiro: mismo flujo, distintas reglas.

Si copias el flujo en cada service → código duplicado. Template Method lo centraliza.

→ Clase abstracta + subclases



56. Template Method: parte 2

→ Record para el contexto: Contiene los datos que se necesitan para la transferencia.

```
TransferContext.java x
1 package com.atlas.bank.atlas.transaction.service;
2
3 import com.atlas.bank.atlas.account.model.Account;
4 import org.springframework.transaction.reactive.TransactionContext;
5
6 import java.math.BigDecimal;
7
8 public record TransferContext(Account from, Account to, BigDecimal amount) {
9 }
10
```

```
TransferContext.java Account.java TransactionProcessor.java x TransferService.java
1 package com.atlas.bank.atlas.transaction.service;
2
3 import com.atlas.bank.atlas.transaction.model.Transaction;
4 import com.atlas.bank.atlas.transaction.repository.TransactionRepository;
5 import lombok.RequiredArgsConstructor;
6 import org.springframework.transaction.annotation.Transactional;
7
8 import java.math.BigDecimal;
9
10 @RequiredArgsConstructor 1 usage 1 inheritor ydochoadev *
11 public abstract class TransactionProcessor<C> {
12
13     // Inyecta el repo porque en cada operación (retiro, trx, depósito)
14     // Para no colocarlo en cada servicio
15     protected final TransactionRepository transactionRepository;
16
17     @Transactional 1 usage ydochoadev
18     public Transaction process(C context) {
19         validate(context);
20         BigDecimal fee = calculateFee(context);
21         execute(context, fee);
22         return save(context, fee);
23     }
24
25     protected abstract void validate(C context); 1 usage 1 implementation
26
27     protected abstract BigDecimal calculateFee(C context); 1 usage 1 implementation
28
29     protected abstract void execute(C context, BigDecimal fee); 1 usage 1 implementation
30
31     protected abstract Transaction save(C context, BigDecimal fee);
32 }
```

57. Template Method: parte 3

➔ Refactorizando la clase TransferService:

```
1 package com.atlas.bank.atlas.transaction.service;
2
3 > import ...
4
16
17 @Service ydochoadev *
18 public class TransferService extends TransactionProcessor<TransferContext> implements ITransferService {
19
20     private final AccountRepository accountRepository; 5 usages
21     private final List<FeeCalculator> feeCalculators; // Se tiene TODA las implementaciones 2 usages
22
23     public TransferService(TransactionRepository transactionRepository, new *
24                             AccountRepository accountRepository,
25                             List<FeeCalculator> feeCalculators) {
26         super(transactionRepository);
27         this.accountRepository = accountRepository;
28         this.feeCalculators = feeCalculators;
29     }
30
31     @Override 1 usage ydochoadev *
32     @Transactional
33     public Transaction execute(Long fromId, Long toId, BigDecimal amount) {
34         // Buscar cuentas
35         Account from = accountRepository.findById(fromId)
36             .orElseThrow(() -> new AccountNotFoundException(fromId));
37         Account to = accountRepository.findById(toId)
38             .orElseThrow(() -> new AccountNotFoundException(toId));
39
40         // process => aplica el patrón Template Method
41         return process(new TransferContext(from, to, amount));
42     }
43
44     @Override 1 usage ydochoadev *
45     protected void validate(TransferContext context) {
46         // Validar que la cuenta esté activa
47         if (!"ACTIVE".equals(context.from().getStatus())) {
48             throw new AccountNotActiveException(context.from().getId(), context.from().getStatus());
49         }
50         if (!"ACTIVE".equals(context.to().getStatus())) {
51             throw new AccountNotActiveException(context.to().getId(), context.to().getStatus());
52         }
53     }
54 }
```

```

protected void validate(TransferContext context) {
    // Validar fondos
    if (context.from().getBalance().compareTo(context.amount()) < 0) {
        throw new InsufficientFundsException(context.from().getId(), context.from().getBalance(), context.amount());
    }
}

@Override @usage @ydochoadev *
protected BigDecimal calculateFee(TransferContext context) {
    // Calcular comisión
    return this.feeCalculators.stream() Stream<FeeCalculator>
        .filter( FeeCalculator fc -> fc.supports(context.from().getType()))
        .findFirst() Optional<FeeCalculator>
        .orElseThrow(() -> new RuntimeException("No hay calculador para el tipo " + context.from().getType()))
        .calculate(context.amount());
}

@Override @usage @ydochoadev *
protected void execute(TransferContext context, BigDecimal fee) {
    // Actualizar saldos
    context.from().setBalance(context.from().getBalance().subtract(context.amount()).subtract(fee));
    context.to().setBalance(context.to().getBalance().add(context.amount()));
    accountRepository.save(context.from());
    accountRepository.save(context.to());
}

@Override @usage @ydochoadev *
protected Transaction save(TransferContext context, BigDecimal fee) {
    // Crear transacción
    Transaction transaction = new Transaction();
    transaction.setType("TRANSFER");
    transaction.setSourceAccountId(context.from().getId());
    transaction.setTargetAccountId(context.to().getId());
    transaction.setAmount(context.amount());
    transaction.setFee(fee);
    transaction.setStatus("EXECUTED");

    return transactionRepository.save(transaction);
}

```

➔ Template Method: Define el esqueleto del algoritmo en una clase base abstracta y deja que las subclases, que la heredan, completen los pasos.

58. Factory Method: Creación desacoplada

➔ Centralizar las decisiones de construcción. No es menos líneas, es menos acoplamiento.

➔ ¿Qué hay dentro de save()?

✗ DECISIONES DISPERSAS

```

TransferService.java

Transaction tx = new Transaction();
tx.setType("TRANSFER");    // string mágico
tx.setSourceAccountId(...);
tx.setTargetAccountId(...);
tx.setAmount(...);
tx.setFee(fee);
tx.setStatus("EXECUTED");  // string mágico

// DepositProcessor repetiría
// lo mismo con "DEPOSIT",
// sin source, fee = ZERO...

```

✓ DECISIONES CENTRALIZADAS

```

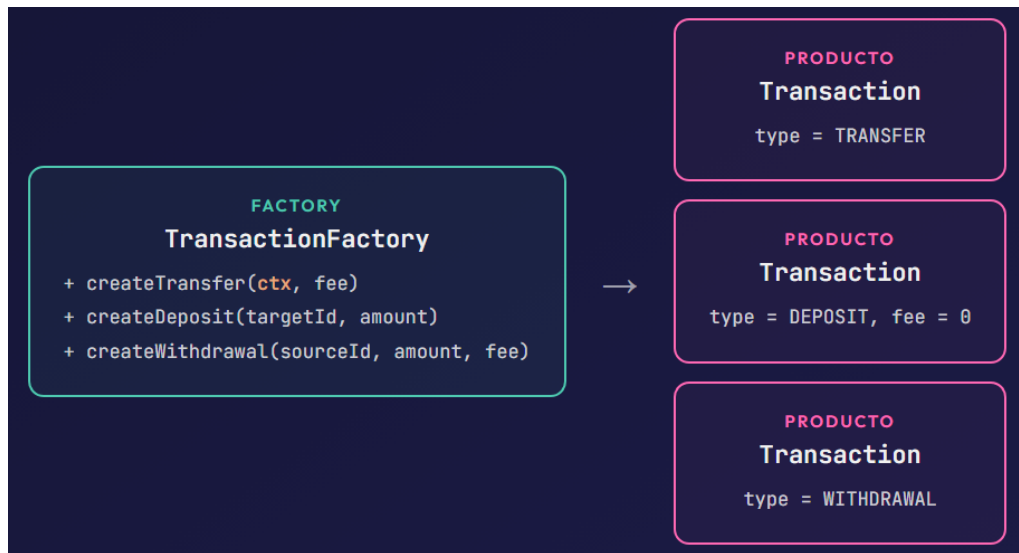
TransferService.java

Transaction tx =
    TransactionFactory
        .createTransfer(ctx, fee);

// El service no sabe:
// - qué string va en type
// - qué status por defecto
// - qué campos mapear
//
// La factory decide todo eso.

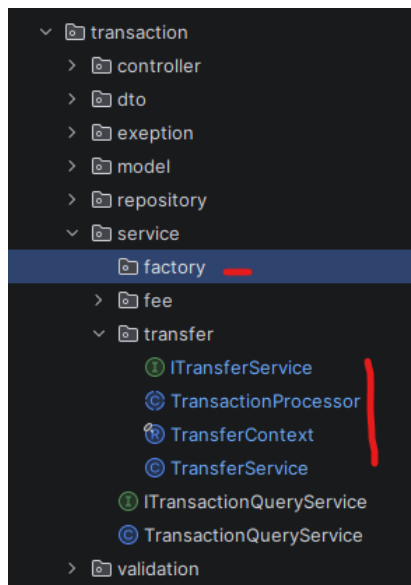
```

→ La factory recibe el contexto



- La factory extrae del **contexto** lo que necesita
- Strings, defaults y reglas de mapeo → **un solo lugar**
- Si Transaction cambia internamente, los services **no se enteran**

→ Crear la carpeta transfer y colocar todo lo relacionado a las transferencias. También crear la carpeta factory:



```

© TransferService.java  © TransactionFactory.java x
1  package com.atlas.bank.atlas.transaction.service.factory;
2
3  import com.atlas.bank.atlas.transaction.model.Transaction;
4  import com.atlas.bank.atlas.transaction.service.transfer.TransferContext;
5
6  import java.math.BigDecimal;
7
8  public class TransactionFactory { 2 usages
9
10 @  public static Transaction createTransfer(TransferContext context, BigDecimal fee) {
11     Transaction transaction = new Transaction();
12     transaction.setType("TRANSFER");
13     transaction.setSourceAccountId(context.from().getId());
14     transaction.setTargetAccountId(context.to().getId());
15     transaction.setAmount(context.amount());
16     transaction.setFee(fee);
17     transaction.setStatus("EXECUTED");
18
19     return transaction;
20 }
21 }

```

➔Modificando la implementación save:

```

© TransferService.java x  © TransactionFactory.java
19  public class TransferService extends TransactionProcessor<TransferContext> implement
46  protected void validate(TransferContext context) {
56      if (context.from().getBalance().compareTo(context.amount()) < 0) {
57          throw new InsufficientFundsException(context.from().getId(), context.fro
58      }
59  }
60
61  @Override @ydochoadev
62  @  protected BigDecimal calculateFee(TransferContext context) {
63      // Calcular comisión
64      return this.feeCalculators.stream() Stream<FeeCalculator>
65          .filter( FeeCalculator fc -> fc.supports(context.from().getType()))
66          .findFirst() Optional<FeeCalculator>
67          .orElseThrow(() -> new RuntimeException("No hay calculador para el t
68          .calculate(context.amount());
69  }
70
71  @Override @ydochoadev
72  @  protected void execute(TransferContext context, BigDecimal fee) {
73      // Actualizar saldos
74      context.from().setBalance(context.from().getBalance().subtract(context.amoun
75      context.to().setBalance(context.to().getBalance().add(context.amount()));
76      accountRepository.save(context.from());
77      accountRepository.save(context.to());
78  }
79
80  @Override @ydochoadev *
81  @  protected Transaction save(TransferContext context, BigDecimal fee) {
82      // Crear transacción
83      Transaction transaction = TransactionFactory.createTransfer(context, fee);
84
85      return transactionRepository.save(transaction);
86  }
87  }
88

```

59. Factory Method: ventaja y limitación

→ Se puede agregar un campo más a la BD (entity): NO realizar el cambio, es solo un ejemplo.

```
Transaction.java x
1 package com.atlas.bank.atlas.transaction.model;
2
3 > import ...
17
18 > @ {...}
24 public class Transaction {
25
26 > @ {...}
28 @
29 private Long id;
30
31 @Column(nullable = false, length = 20)
32 private String type; // DEPOSIT, WITHDRAWAL, ...
33
34 @Column(name = "source_account_id")
35 private Long sourceAccountId;
36
37 @Column(name = "target_account_id")
38 private Long targetAccountId;
39
40 @Column(nullable = false)
41 private BigDecimal amount;
42
43 @Column(nullable = false)
44 private BigDecimal fee;
45
46 @Column(nullable = false, length = 20)
47 private String status; // PENDING, EXECUTED, ...
48
49 @Column(name = "create_at", nullable = false,
50 private LocalDateTime createdAt;
51
52 private String createdBy;
53
54 @PrePersist @ ydochoadev
55 public void prePersist() {...}
56
57 }
```

Y en el Factory:

```
Transaction.java TransactionFactory.java x
1 package com.atlas.bank.atlas.transaction.service.factory;
2
3 import com.atlas.bank.atlas.transaction.model.Transaction;
4 import com.atlas.bank.atlas.transaction.service.transfer.TransferContext;
5
6 import java.math.BigDecimal;
7
8 public class TransactionFactory { 2 usages @ ydochoadev *
9
10 @ public static Transaction createTransfer(TransferContext context, B
11     Transaction transaction = new Transaction();
12     transaction.setType("TRANSFER");
13     transaction.setCreatedBy("SYSTEM");
14     transaction.setSourceAccountId(context.from().getId());
15     transaction.setTargetAccountId(context.to().getId());
16     transaction.setAmount(context.amount());
17     transaction.setFee(fee);
18     transaction.setStatus("EXECUTED");
19
20     return transaction;
21
22 }
```

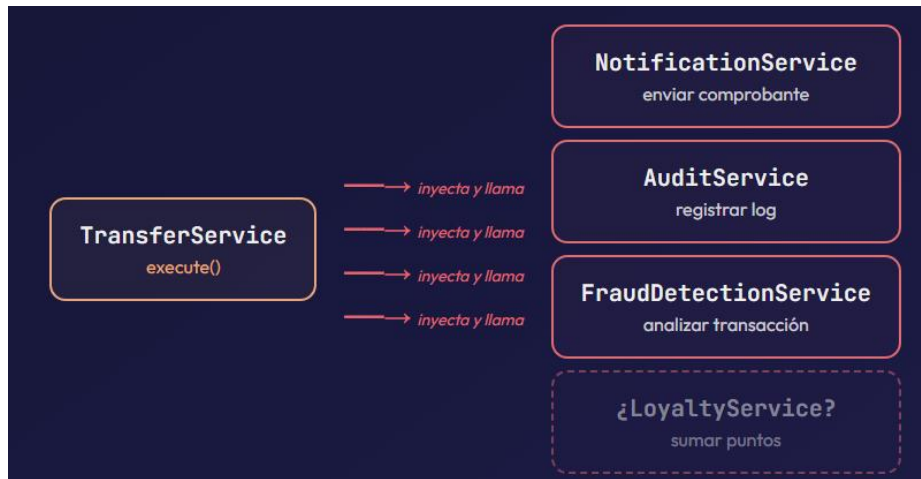
** FACTORY NO puede resolver todo. Cuando hay un nuevo dato que debe enviar el cliente (por ejemplo “description”), se tiene que cambiar varios archivos: transfer entity, transfer request, factory, transfer context.

La Factory absorbe cambios internos: datos por default, reglas de mapeo que no depende del cliente o usuario.

60. Observer: Eventos de aplicación

→ Cuando algo pasa, los interesados se enteran y reaccionan. Sin que el emisor los conozca.

→ Side effects acoplados



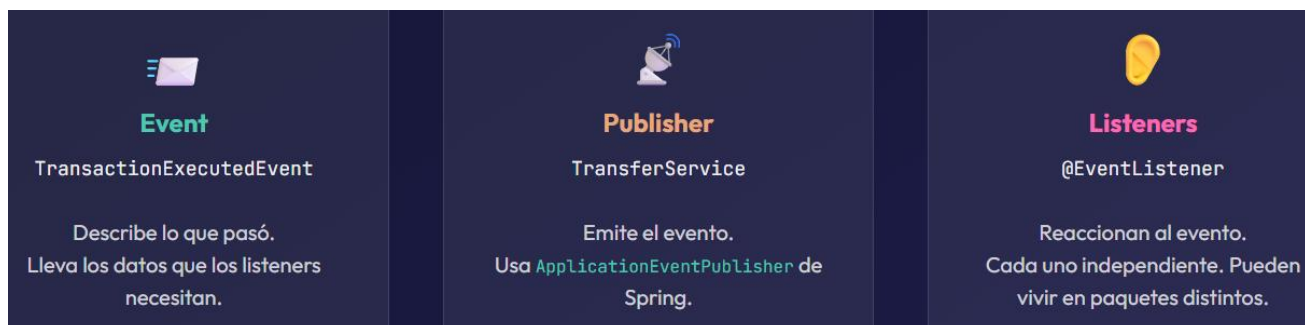
Cada efecto secundario = una dependencia más en TransferService.

→ Solución: Publicar y reaccionar



** Spring ya tiene implementado este patrón, solo sería usarlo.

→ Los tres participantes



61. Observer en Spring: ApplicationEventPublisher

➔Evento => Data necesaria de la trx. Es un objeto que describe algo que pasó y que no debe ser modificable y lleva todos los datos que los listener necesita.

```
TransactionExecutedEvent.java x
1 package com.atlas.bank.atlas.transaction.service.event;
2
3 import java.math.BigDecimal;
4
5 public record TransactionExecutedEvent( no usages
6     Long transactionId, no usages
7     String type, no usages
8     Long sourceAccount, no usages
9     Long targetAccount, no usages
10    BigDecimal amount, no usages
11    BigDecimal fee no usages
12 ) {
13 }
14
```

➔Publisher => En el service, publicar dicho evento luego de ejecutar la transferencia.

```
TransactionExecutedEvent.java TransferService.java x
1 package com.atlas.bank.atlas.transaction.service.transfer;
2
3 > import ...
19
20 @Service @ydochoadev *
21 public class TransferService extends TransactionProcessor<TransferContext> implements ITransferService {
22
23     private final AccountRepository accountRepository;
24     private final List<FeeCalculator> feeCalculators; // Se tiene TODA las implementaciones
25     private final ApplicationEventPublisher eventPublisher;
26
27     public TransferService(TransactionRepository transactionRepository, @ydochoadev *
28         AccountRepository accountRepository,
29         List<FeeCalculator> feeCalculators,
30         ApplicationEventPublisher eventPublisher) {
31         super(transactionRepository);
32         this.accountRepository = accountRepository;
33         this.feeCalculators = feeCalculators;
34         this.eventPublisher = eventPublisher;
35     }
36 }
```

En el método execute:

```
@Override @ydochoadev *
@Transactional
public Transaction execute(Long fromId, Long toId, BigDecimal amount) {
    // Buscar cuentas
    Account from = accountRepository.findById(fromId)
        .orElseThrow(() -> new AccountNotFoundException(fromId));
    Account to = accountRepository.findById(toId)
        .orElseThrow(() -> new AccountNotFoundException(toId));

    // process => aplica el patrón Template Method
    Transaction transaction = process(new TransferContext(from, to, amount));
    // Lanzar el evento
    eventPublisher.publishEvent(new TransactionExecutedEvent(
        transaction.getId(),
        transaction.getType(),
        transaction.getSourceAccountId(),
        transaction.getTargetAccountId(),
        transaction.getAmount(),
        transaction.getFee()
    ));
    return transaction;
}
```

→ Listeners: Se crea una carpeta para los listeners. El primero será para notificaciones:

```
TransactionExecutedEvent.java  TransferService.java  NotificationListener.java x
1 package com.atlas.bank.atlas.transaction.service.listener;
2
3 import com.atlas.bank.atlas.transaction.service.event.TransactionExecutedEvent;
4 import lombok.extern.slf4j.Slf4j;
5 import org.springframework.context.event.EventListener;
6 import org.springframework.stereotype.Component;
7
8 @Component
9 @Slf4j
10 public class NotificationListener {
11
12     @EventListener
13     public void onTransactionExecuted(TransactionExecutedEvent event) {
14         log.info("Enviando comprobante de {} por S/ {} - transacción #{}",
15             event.type(), event.amount(), event.transactionId());
16     }
17 }
18
```

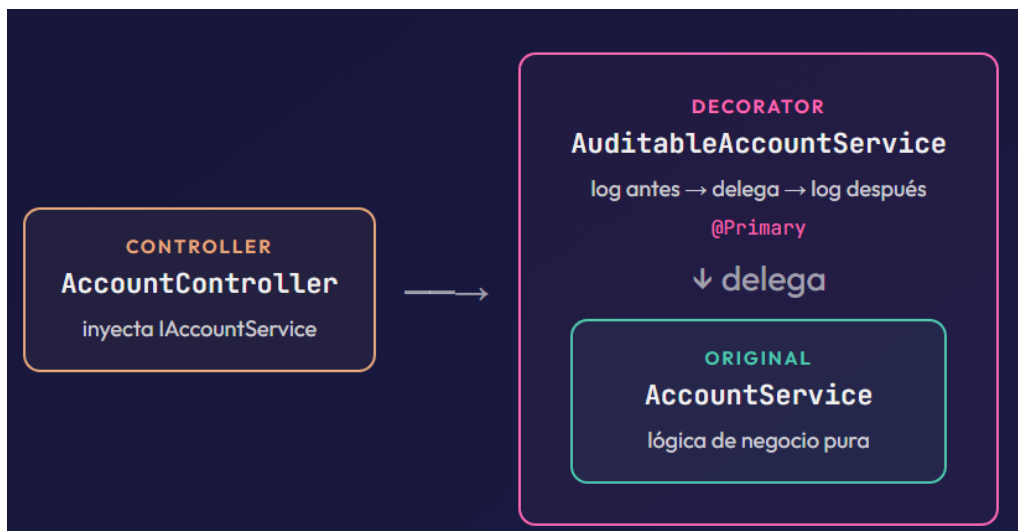
→ Creando nuevo listener:

```
TransactionExecutedEvent.java  TransferService.java  NotificationListener.java  AuditListener.java x
1 package com.atlas.bank.atlas.transaction.service.listener;
2
3 import com.atlas.bank.atlas.transaction.service.event.TransactionExecutedEvent;
4 import lombok.extern.slf4j.Slf4j;
5 import org.springframework.context.event.EventListener;
6 import org.springframework.stereotype.Component;
7
8 @Component
9 @Slf4j
10 public class AuditListener {
11
12     @EventListener
13     public void onTransactionExecuted(TransactionExecutedEvent event) {
14         log.info("Registrando auditoria - {} de cuenta #{} a cuenta #{} por S/ {}",
15             event.type(), event.sourceAccountId(), event.targetAccountId(), event.amount());
16     }
17 }
18
```

** TransferService publica el evento de tipo “TransactionExecutedEvent” y los listener escuchan ese evento, ya que están con el decorador @EventListener y en su parámetro de entrada está la clase evento “TransactionExecutedEvent”.

62. Decorador: Enriquecer sin modificar 1

→ Agregar comportamiento a un objeto sin modificar su código. Envolver, no heredar.



** El controller no sabe que existe el decorator. AccountService no sabe que está siendo decorado.

→ Spring AOP (Aspect Oriented Programming): Decorators que ya usás

@Transactional	@Cacheable	@Async
Abre transacción antes del método. Commit si sale bien. Rollback si hay excepción.	Verifica el caché antes de ejecutar. Si hay resultado cacheado, lo devuelve. Si no, ejecuta y cachea.	Ejecuta el método en otro thread. El caller no espera el resultado. Procesamiento asíncrono.
Proxy que envuelve el bean original	Proxy que envuelve el bean original	Proxy que envuelve el bean original

63. Decorator: Enriquecer sin modificar 2

→ Decorar el AccountService para colocar logs:

→ Cuando se ejecuta en el controller el servicio de accountService, lo envía a la implementación que tiene el decorador @Primary, ya que este será el que realice el trabajo.

```
@RestController @ydochoadev
@RequestMapping("/api/v1/accounts")
@AllArgsConstructor
public class AccountController {

    private final IAccountService accountService;
    private final AccountMapper accountMapper;

    @PostMapping @ydochoadev
    public ResponseEntity<AccountResponse> create(@Valid I
```

```
AuditableAccountService.java x AccountController.java
4 import lombok.extern.slf4j.Slf4j;
5 import org.springframework.beans.factory.annotation.Qualifier;
6 import org.springframework.context.annotation.Primary;
7 import org.springframework.stereotype.Component;
8
9 import java.util.List;
10
11 @Slf4j
12 @Component
13 @Primary
14 public class AuditableAccountService implements IAccountService {
15
16     private final IAccountService delegate; 2 usages
17
18     public AuditableAccountService(@Qualifier("accountService") IAccountService delegate) {
19         this.delegate = delegate;
20     }
21
22     @Override
23     public Account create(Account account) {
24         log.info("Creando cuenta - número {}, titular: {}", account.getAccountNumber(), account.getOwner());
25         Account created = delegate.create(account);
26         log.info("Cuenta creada exitosamente - ID: {}", account.getId());
27
28         return created;
29     }
30
31     @Override
32     public List<Account> getAllAccounts() {
33         return delegate.getAllAccounts();
34     }
35
36     @Override
37     public Account getAccountById(Long id) {
38         return delegate.getAccountById(id);
39     }
40
41     @Override
42     public void deleteAccount(Long id) {
43         delegate.deleteAccount(id);
44     }
45 }
```

** En el método constructor:

```
public AuditableAccountService(@Qualifier("accountService") IAccountService delegate) {  
    this.delegate = delegate;  
}
```

@Qualifier("accountService") ➔ Busca en los beans el objeto “accountService” y delega el trabajo a esa implementación.

```
AuditableAccountService.java | AccountService.java x | AccountController.java  
1 package com.atlas.bank.atlas.account.service;  
2  
3 > import ...  
10  
11 @Service @ydochoadev  
12 @RequiredArgsConstructor  
13 public class AccountService implements IAccountService {  
14  
15     private final AccountRepository accountRepository;  
16  
17     @Override @ydochoadev  
18     public Account create(Account account) { return accountRepository.save(account); }  
21  
22     @Override 1 usage @ydochoadev  
23     public List<Account> findAll() { return accountRepository.findAll(); }  
26  
27     @Override 1 usage @ydochoadev  
28     public Account findById(Long id) {  
29         return accountRepository.findById(id).orElseThrow(() -> new AccountNotFoundException(id));  
30     }  
31 }  
32
```

64. Spring AOP: el decorador que ya estás usando

➔ ¿Qué es AOP?

Aspect-Oriented Programming: un mecanismo para interceptar la ejecución de métodos y agregar comportamiento antes, después o alrededor, sin tocar el código original.

Cuando Spring encuentra ciertas anotaciones, genera un proxy en runtime que envuelve al bean. Ese proxy es un Decorator automático.



➔ No toda @ es AOP

ENVUELVEN LA EJECUCIÓN ➔ AOP	CONFIGURAN EL BEAN ➔ NO ES AOP
Generan un proxy que intercepta el método.	Le dicen a Spring cómo registrar o inyectar.
@Transactional @Cacheable @Async @Retryable	@Component @Service @Primary @Qualifier @Order
Agregan comportamiento alrededor del método sin modificar su código.	No interceptan nada. Solo configuran cómo se arma el contexto.

** Boilerplate => Cuando se crea un decorador, se tiene que colocar todas las implementaciones y esto genera código doble, ya que en no todas las implementaciones se requiere realizar algo.

➔ ¿Cuándo cada uno?

Decorator manual	Decorator AOP
✓ Lógica específica de negocio	✓ Genérico, aplica a muchos beans
✓ Visible en el código fuente	✓ Cero boilerplate
✓ Fácil de debuggear	✓ Una anotación, nada más
✗ Boilerplate si la interfaz crece	✗ Invisible, más difícil de debuggear
AuditableAccountService	@Transactional, @Cacheable